Lab 7: Fuzzing for Input Validation Bugs

Repository: L6_L7_20_food_delivery (Lab 7 folder). Application: food_delivery.py. Tests: test_food_delivery.py.

Step 2: Running the buggy application

I ran the original food_delivery.py with normal inputs at the first two prompts and the coupon code FREE50 at the third. parse_order and delivery_eta worked. coupon_discount crashed with an IndexError: FREE50 does not contain "SAVE", so split('SAVE') returns a list with one element and index [1] does not exist.

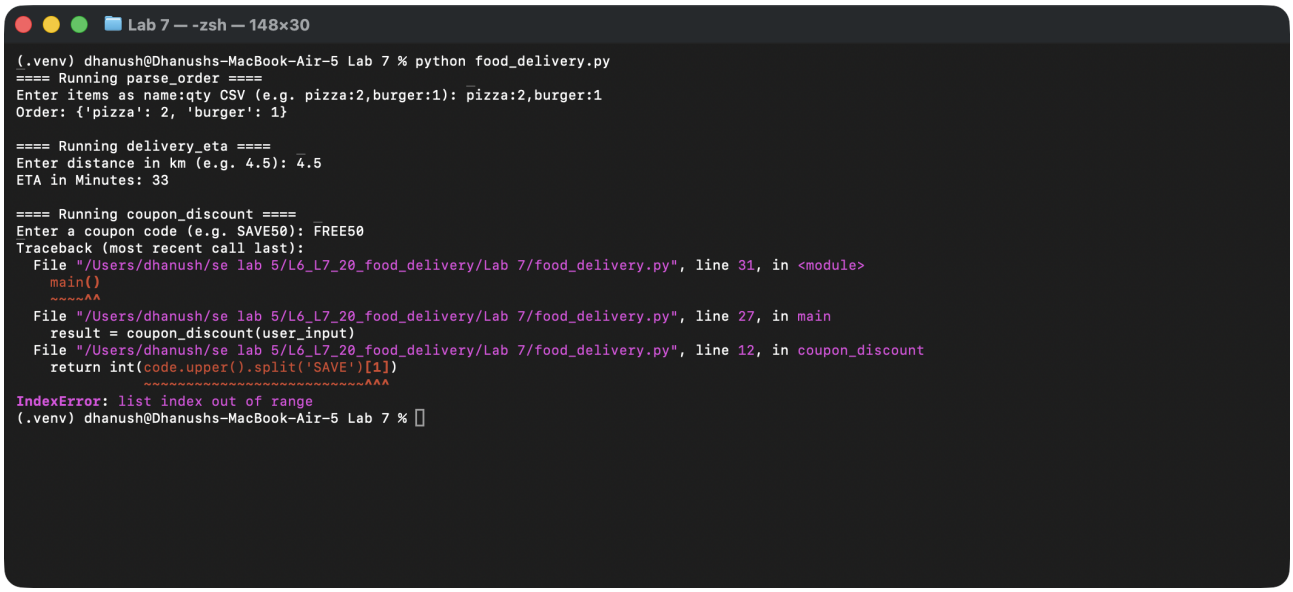


Figure 1: python food_delivery.py on the original code.

Step 3: Writing the fuzz tests with Hypothesis

test_food_delivery.py has seven tests. Three are no-crash tests that pass random text or None to each function. The other four check properties:

- a valid order string parses back to the same dictionary;
- delivery_eta never crashes on any float and never returns less than the 15-minute base time;
- a valid SAVE code returns its number in upper or lower case;
- a negative code such as SAVE-1 never gives a negative discount.

Step 4: Running the fuzz tests on the buggy application

Five of the seven tests failed. Hypothesis found these problems:

- parse_order crashes on None (AttributeError), on an item without a colon (ValueError: not enough values to unpack) and on an empty quantity (ValueError from int()).
- delivery_eta crashes on None (TypeError) and on an empty string (ValueError). It returns 11 minutes for a distance of -1.0, and a very large distance such as 4.49e307 makes int() raise OverflowError.

- coupon_discount crashes on None (AttributeError) and on any text without SAVE (IndexError). SAVE-1 gives a discount of -1.

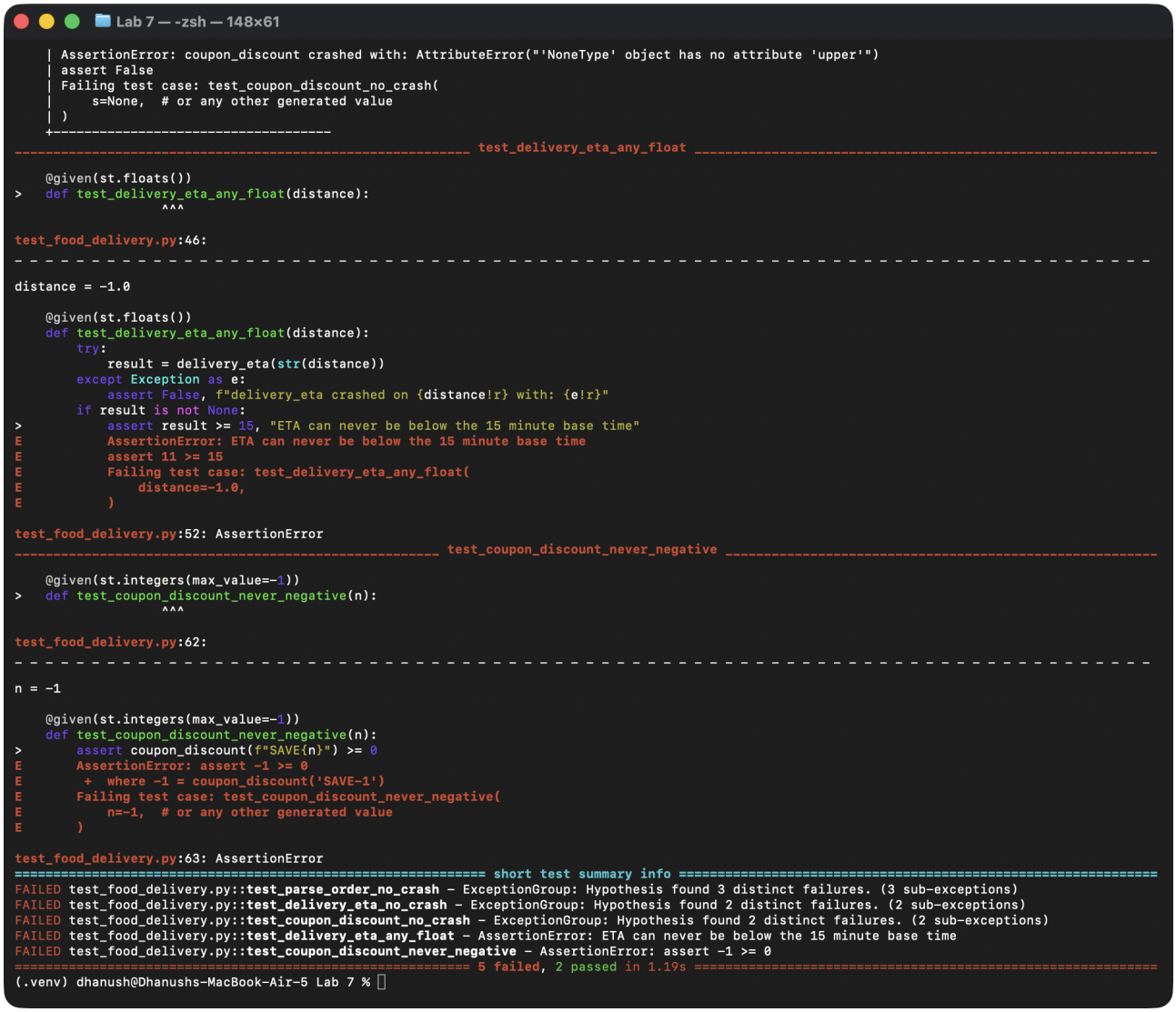


Figure 2: pytest test_food_delivery.py on the original code (end of the output with the failure summary).

Step 5: Fixing the application

parse_order now returns an empty dictionary for anything that is not a string. It skips items with no colon, no name, a non-numeric quantity or a quantity below 1. delivery_eta returns None when the value is not a number, is negative or gives an infinite result. coupon_discount returns 0 unless the code is SAVE followed only by digits. main() prints a short message instead of None or 0.

The screenshot shows the fixed script run twice: once with the same inputs as Figure 1 and once with invalid inputs. Neither run crashes.

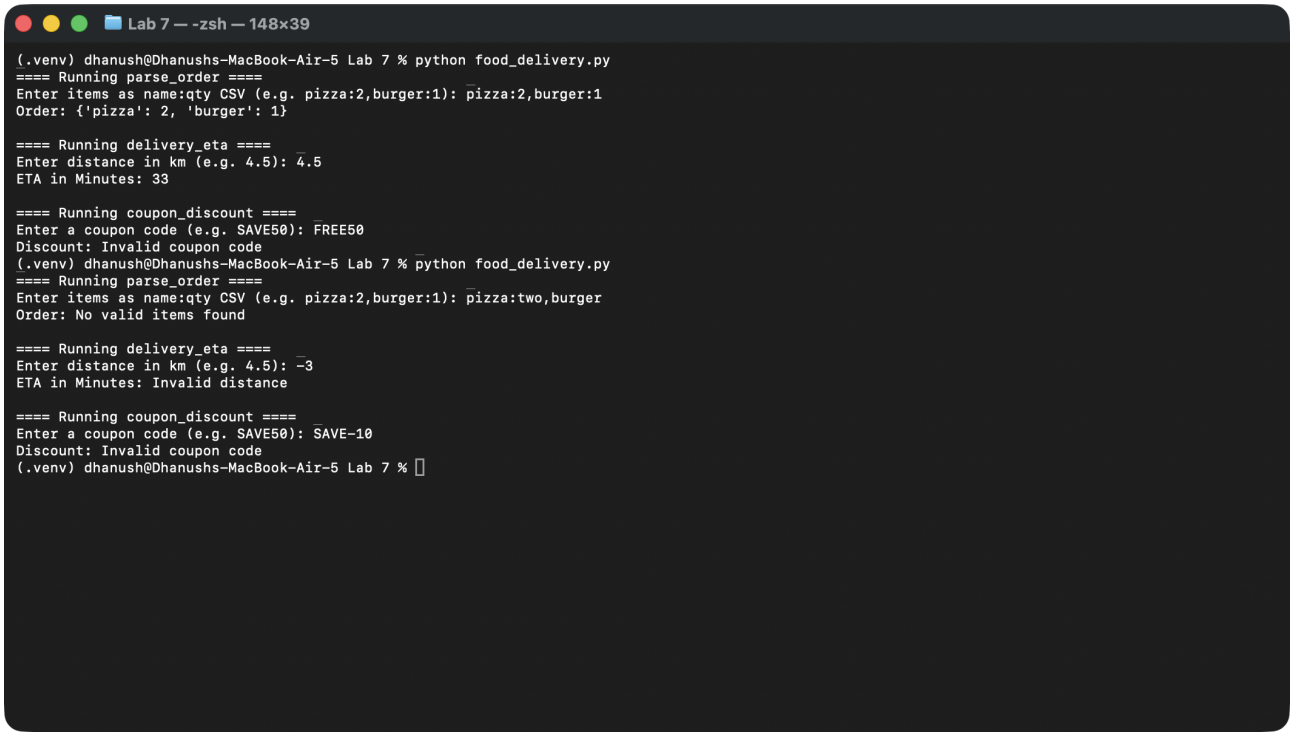


Figure 3: python food_delivery.py on the fixed code.

Step 6: Running the fuzz tests on the fixed application

All seven tests pass against the fixed food_delivery.py.

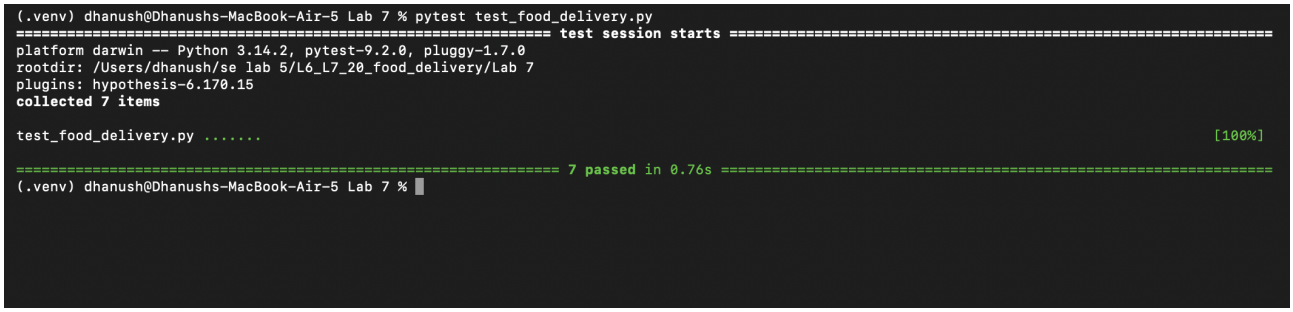


Figure 4: pytest test_food_delivery.py on the fixed code.